

Redis Beyond Caching: Use Cases Research

Research reference · 2026-08-24

Redis Use Cases Beyond Caching — Research Notes

Goal: map what enterprises actually use Redis for beyond our current trio (cache, identifiers, distributed locks) — with when/why/how per pattern.

Compiled Aug 2026. Sources at bottom.

TL;DR

Redis is not a cache with extras — it's an **in-memory data-structures server**. Every advanced use case is really "a data structure + an access pattern":

Problem	Data structure	Core commands
Rate limiting	String counters / Sorted Set / Hash	<code>INCR + EXPIRE</code> , <code>ZADD / ZREMRANGEBYSCORE</code> , Lua
Leaderboards / ranking	Sorted Set	<code>ZADD</code> , <code>ZINCRBY</code> , <code>ZRANGE</code> , <code>ZREVRANK</code>
Reliable queues / event streams	Stream	<code>XADD</code> , <code>XREADGROUP</code> , <code>XACK</code> , <code>XAUTOCLAIM</code>
Fire-and-forget broadcast	Pub/Sub	<code>PUBLISH</code> , <code>SUBSCRIBE</code>
Lightweight queues	List	<code>LPUSH</code> , <code>BRPOP</code>
Unique counting	HyperLogLog	<code>PFADD</code> , <code>PFCOUNT</code> (~0.81% std error, ~12KB/set)
Binary activity flags (DAU etc.)	Bitmap	<code>SETBIT</code> , <code>BITCOUNT</code>
Sessions	String/Hash + TTL	<code>SET ... EX</code> , <code>HSET</code>
Nearby / radius search	Geospatial (Sorted Set under the hood)	<code>GEOADD</code> , <code>GEOSEARCH</code>
Mutual exclusion	String + <code>SET NX</code>	<code>SET NX PX</code> , Lua compare-and-delete

Cross-cutting enablers: **Lua scripting (EVAL)** makes read-decide-write atomic, TTL gives automatic cleanup, pipelining amortizes round trips.

1. Rate limiting

Why Redis: every app instance must agree on one shared counter; Redis gives sub-millisecond allow/deny decisions from a shared store.

Five algorithms (all wrapped in Lua for atomicity):

Algorithm	Memory/client	Accuracy	Burst behavior	Best for
Fixed window	1 key	Approximate	2× burst at boundaries	Simple API limits, login throttling
Sliding window log	O(n) entries	Exact	None	High-value APIs, audit trails
Sliding window counter	2 keys	Near-exact	Smoothed	General-purpose APIs
Token bucket	1 hash	Exact	Controlled bursts	Bursty traffic w/ average cap
Leaky bucket	1 hash	Exact	No bursts (drain)	Strict policing/shaping

Key mechanics:

- Fixed window = `INCR key:windowTs` + `EXPIRE` — but only atomically inside Lua, else a crash between the two leaves a permanent key (and concurrent requests can bypass limits — TOCTOU).
- Sliding log = Sorted Set of timestamps; prune with `ZREMRANGEBYSCORE`, count with `ZCARD`.
- Token bucket = Hash `{tokens, last_refill}`; refill computed from elapsed time, capped at capacity.
- Cluster note: multi-key scripts need hash tags `{base}:w1`, `{base}:w2` to co-locate slots.
- Use `redis.call('TIME')` inside scripts to avoid app-server clock drift.

2. Leaderboards & ranking (Sorted Sets)

Why Redis: ranking in SQL is `ORDER BY` over the whole table (O(N), seconds at millions of rows); caches go stale because scores change on every action.

- One sorted set per board; member = entity ID, score = value ranked on.
- `ZINCRBY` updates scores atomically in place — no read-modify-write, no cache invalidation.
- `ZREVRANK` gives exact global rank; `ZRANGE ... REV LIMIT` serves top-N and "around me" neighborhoods in one call.
- Per-window keys (daily/weekly/monthly) + `EXPIRE` self-clean; `ZUNIONSTORE` aggregates windows without app-side sorting.
- Scale reference: millions of ranked members $\approx$ 500MB RAM. Companion Hash holds metadata so the set stays light.
- Real users: Scopely (mobile gaming leaderboards), Freshworks.

3. Messaging: Pub/Sub vs Lists vs Streams

Feature	Pub/Sub	Lists	Streams
Persistence	No (fire-and-forget)	Yes	Yes
Consumer groups	No	No	Yes
Replay history	No	No	Yes
Acknowledgment	No	Manual	Built-in (XACK)
Multiple consumers	Broadcast	Competing	Both
Delivery guarantee	At-most-once	At-most-once	At-least-once

Streams mechanics (the important part):

- Append-only log; consumers move cursors, reads are non-destructive (vs Lists where pop removes).
- Consumer group delivers each message to exactly one consumer; delivered-but-unacked messages sit in the **Pending Entries List (PEL)**.
- Crash recovery: XCLAIM / XAUTOCLAIM (6.2+) transfers orphaned pending messages after an idle threshold — decentralized work stealing.
- Correct consumer startup is **two-phase**: drain own PEL with ID 0 first, then read new messages with > . Consumers that only ask for > create zombie messages stuck in the PEL forever.
- Poison pills: track times-delivered (from XPENDING/XAUTOCLAIM); route to a DLQ after N attempts — Redis has no built-in DLQ, it's application code.
- Memory: don't XDEL after ACK (Radix-tree macro-nodes fragment — "Swiss cheese"); XACK for state, XTRIM MAXLEN ~ N for retention (approximate trim frees whole macro-nodes cheaply).
- Blocking reads: dedicated connections + short BLOCK timeouts, else you exhaust the connection pool.
- Lag signal: growing PEL while throughput looks healthy = ACK failures or processing bottleneck, not delivery problems.

Streams vs Kafka: RAM (hours/days retention) vs disk (months/years); sub-ms vs low-ms latency; manual vs native partitioning; manual (XAUTOCLAIM) vs automatic rebalancing; trivial vs heavy ops. Rule: Streams for inter-service events + active job queues when you already run Redis; Kafka for long-retention event sourcing and very high throughput.

Real users: Freshworks (persistent background-job store), microservices inter-service communication patterns (official howtos).

4. Counting at scale

- **Counters:** `INCR` / `INCRBY` at multiple granularities (minute/hour/day keys) with staggered TTLs — pipeline the writes.
- **HyperLogLog:** unique visitors/sessions with ~12KB per set regardless of cardinality, ~0.81% standard error. `PFADD` / `PFCOUNT` / `PFMERGE` .
- **Bitmaps:** binary states (did user X act today?) — `SETBIT user:{day} {userId} 1` ; `BITCOUNT` for DAU; AND/OR across days for retention cohorts.
- Real users: Freshworks (user-analytics frontend with HLL + bitmaps + sets).

5. Session store

Hash/String + TTL; billions of field-value pairs at sub-ms; auth microservice pattern (Freshworks split sessions out of a Rails monolith as its first microservice). Why Redis over DB-backed sessions: load removal from primary DB + expiration semantics built in.

6. Geospatial

`GEOADD` stores coordinates as 52-bit geohash integers inside a Sorted Set; `GEOSEARCH` (6.2+) does nearest-N and radius queries at $\sim O(N+\log(M))$. Groupon production case: millions of geo queries/minute ("nearest deal", "deals within radius"), scaled horizontally by sharding keys across cluster nodes; keep keys small and partitioned, watch radius growth (bounding-box size drives cost).

7. Distributed locks — the part we already do, done right

Single-instance recipe: acquire `SET key uniqValue NX PX ttl` ; release via Lua compare-and-delete (never bare `DEL` — you might delete someone else's lock after your TTL expired).

The Redlock controversy (must-know):

- Redlock: acquire on N=5 independent masters, majority wins. Claims fault-tolerant mutual exclusion.
- Kleppmann's critique: (a) no fencing tokens — a client paused past TTL can still write after another client acquired; (b) safety depends on timing assumptions (bounded delay/pause/clock drift) — a synchronous-system assumption that fails exactly when things go wrong; verdict: "neither fish nor fowl" — too heavy for efficiency locks, not safe enough for correctness locks.
- antirez's rebuttal: validity-time rechecks during acquisition mitigate timing issues; fsync-always or delayed restarts handle persistence races; agrees monotonic-clock note is fair.
- Both agree on the practical resolution:

- **Efficiency locks** (avoid duplicate work): single-instance `SET NX PX` + compare-and-delete is fine; document that it's approximate.

- **Correctness locks** (violation = corruption): use ZooKeeper/etcd/consensus or a transactional DB, and enforce **fencing tokens** (monotonic number checked by the storage layer, reject stale-token writes).

- Redis docs themselves now carry the disclaimer: implement fencing tokens; wall-clock shifts can double-acquire.

8. When NOT to reach for Redis

- Durable system-of-record (RDB/AOF are best-effort, not CP guarantees).
- Long-retention event history / replay across months → Kafka.
- Very large datasets (RAM economics) unless using tiered options.
- Hot-key / big-value hazards: single-threaded core means one huge key or hot key throttles everything; blocking commands (`KEYS` , big `SMEMBERS` , unbounded `BRPOP` loops on shared pools) stall all clients.
- Multi-key Lua in cluster mode requires hash-tag co-location — design keys accordingly.

Sources

- Kleppmann, [How to do distributed locking](#) · antirez, [Is Redlock safe?](#) · [Redis distributed locks docs](#) · antirez, [Reliable locks proposal](#)
- Rate limiting: [Redis rate limiter guide](#) · [5 rate limiters tutorial](#) · [antirez rate-limiting patterns](#)
- Streams: [Streams data type docs](#) · [Consumer group patterns](#) · [Interservice communication howto](#) · [Event sourcing patterns](#)
- Use-case catalog: [Redis industry use cases](#) · [Leaderboard use case](#) · [Real-time analytics guide](#) · [Groupon geospatial at scale](#) · [Severalnines Redis intro](#)